

BLOCKCHAIN LABORATORY

Experiment No. 3

Create a Cryptocurrency using Python

Name: Nikhil Sunchu

Roll No.: 31

Class/section D20A

Date _____

1. Aim

Create a Cryptocurrency using Python and perform mining in the Blockchain created.

2. Theory

2.1. 1. Blockchain Overview

Blockchain is a distributed and decentralized ledger that stores information in a series of linked blocks. Each block contains transaction data, a timestamp, the previous block's hash, and its own unique hash (a digital fingerprint). Once data is recorded, it becomes immutable, because altering one block would require recalculating all subsequent blocks.

2.2. 2. Mining

Mining is the process of collecting pending transactions into a block, solving a computational puzzle (Proof-of-Work) to find a valid hash, and adding the new block to the blockchain before broadcasting it to all connected peers. Miners are rewarded with cryptocurrency for successfully mining a block.

2.3. 3. Multi-Node Blockchain Network

In this lab we simulate three independent blockchain nodes running on ports 5001, 5002, and 5003. Each node runs on a separate port, maintains its own copy of the blockchain, and can connect with peers to share and validate blocks.

2.4. 4. Consensus Mechanism

We use the *Longest Chain Rule*: if multiple versions of the chain exist, the longest valid chain is chosen. This ensures that all nodes agree on a single transaction history.

2.5. 5. Transactions and Mining Reward

Each transaction has a sender, a receiver, and an amount. When a block is mined, the pending transactions are added to the block, and a reward transaction is added automatically to pay the miner.

2.6. 6. Chain Replacement

When `/replace_chain` is called, the node requests the chains from all its peers. If it finds a longer and valid chain, it replaces its own chain with it, keeping the blockchain consistent across all nodes.

3. Tools and Libraries Used

- Python 3.x
- Flask – web framework for the API endpoints (`pip install Flask`)
- Requests – HTTP communication between nodes (`pip install requests==2.18.4`)
- Postman – for testing the API requests
- Python standard libraries: `datetime`, `jsonify`, `hashlib`, `uuid4`, `urlparse`, `request`

4. Tasks Performed

4.1. Task 1 – Download the Code

The files `hadcoin_node_5001.py`, `hadcoin_node_5002.py`, and `hadcoin_node_5003.py` were downloaded. Each implements a full cryptocurrency node – the `Blockchain` class plus a Flask API – differing only in the port number and the mining-reward receiver name.

4.2. Task 2 – Install the Required Packages

The `requests` library was installed in the virtual environment created in Lab 2, together with Flask:

```
pip install Flask
pip install requests==2.18.4
```

4.3. Task 3 – Run the Three Nodes in Separate Terminals

The three node files were run in three separate terminals, so each node listens on its own port (5001, 5002, 5003) and maintains its own copy of the blockchain.

4.4. Task 4 – Connect the Nodes (POST `/connect_node`)

The peers were registered from each node by invoking `connect_node()` with a POST request containing the node addresses in JSON. `add_node()` parses each URL with `urlparse()` and stores the network location (e.g. `127.0.0.1:5002`) in the node set.

```
1 def add_node(self, address):
2     parsed_url = urlparse(address)
3     self.nodes.add(parsed_url.netloc)
4
5 @app.route('/connect_node', methods=['POST'])
6 def connect_node():
7     json = request.get_json()
8     nodes = json.get('nodes')
9     if nodes is None:
10         return 'No node', 400
11     for node in nodes:
```

```

12     blockchain.add_node(node)
13     return jsonify({'message': 'All the nodes are now connected.',
14                       'total_nodes': list(blockchain.nodes)}), 201

```

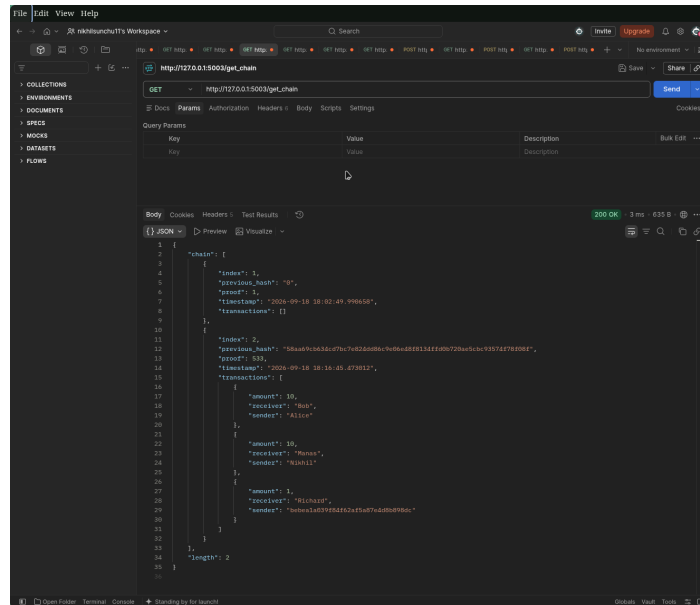


Fig. 1 – Connecting the nodes (POST /connect_node)

4.5. Task 5.1 – Add Transactions (POST /add_transaction)

Transactions were added through a POST request containing the sender, receiver, and amount. `add_transaction()` appends the transaction to the pending pool and returns the block index in which it will be mined.

```

1 def add_transaction(self, sender, receiver, amount):
2     self.transactions.append({'sender': sender,
3                               'receiver': receiver,
4                               'amount': amount})
5     return self.get_previous_block()['index'] + 1
6
7 @app.route('/add_transaction', methods=['POST'])
8 def add_transaction():
9     json = request.get_json()
10    if not all(key in json for key in ['sender', 'receiver', 'amount']):
11        return 'Some elements of the transaction are missing', 400
12    index = blockchain.add_transaction(json['sender'],
13                                      json['receiver'],
14                                      json['amount'])
15    return jsonify({'message':
16                    f'This transaction will be added to Block {index}'}), 201

```

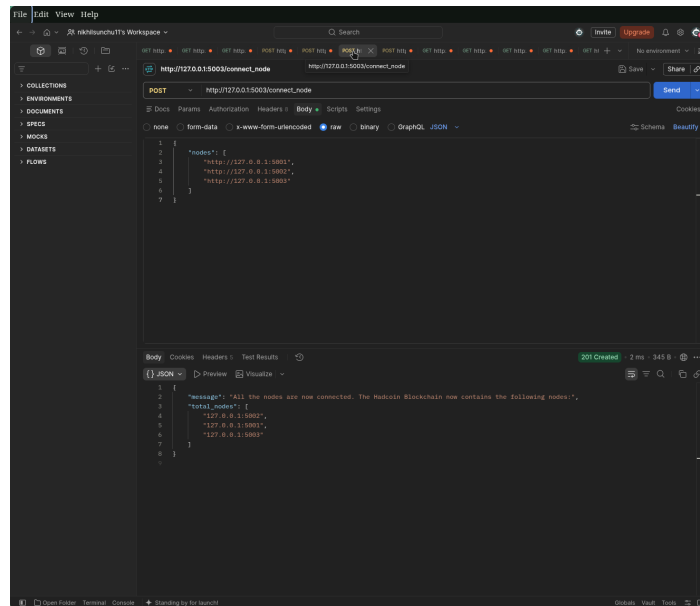


Fig. 2 – Adding a transaction (POST /add_transaction)

4.6. Task 5.2 – Mining (GET /mine_block)

Mining was performed with `mine_block()`: the node takes the proof of the previous block, solves the Proof-of-Work puzzle (hash starting with 0000), and automatically adds a reward transaction of 1 Hadcoin to the miner before creating the block.

```

1 @app.route('/mine_block', methods=['GET'])
2 def mine_block():
3     previous_block = blockchain.get_previous_block()
4     proof = blockchain.proof_of_work(previous_block['proof'])
5     previous_hash = blockchain.hash(previous_block)
6     blockchain.add_transaction(sender=node_address,
7                               receiver='Richard', amount=1)
8     block = blockchain.create_block(proof, previous_hash)
9     return jsonify({'message': 'Congratulations, you just mined a block!',
10                    'index': block['index'],
11                    'timestamp': block['timestamp'],
12                    'proof': block['proof'],
13                    'previous_hash': block['previous_hash'],
14                    'transactions': block['transactions']}), 200

```

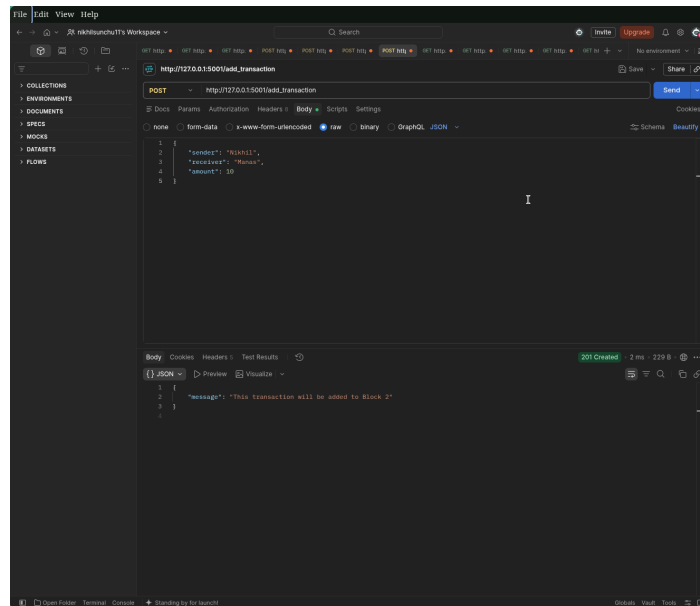


Fig. 3 – Mining a block with the added transaction

4.7. Task 5.3 – Fetch the Chain (GET /get_chain)

The complete blockchain of a node was fetched with `get_chain()`, which returns the chain together with its length.

```
@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({'chain': blockchain.chain,
                    'length': len(blockchain.chain)}), 200
```

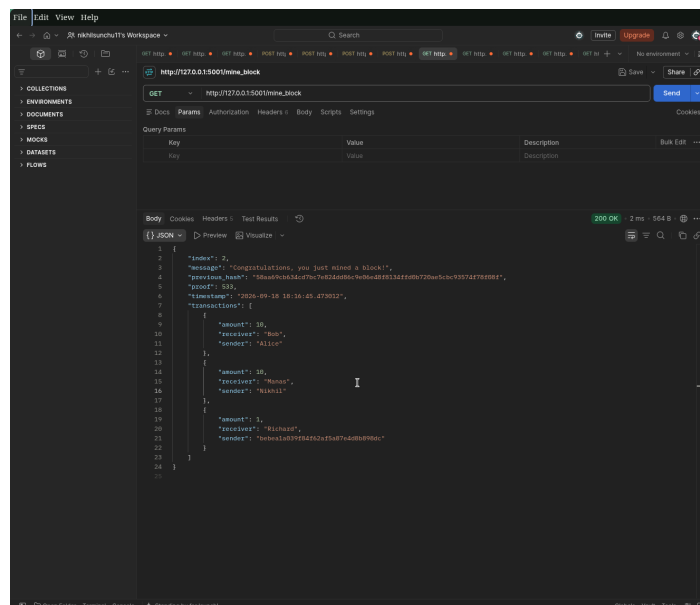


Fig. 4 – Fetching the chain with the mined block

4.8. Task 5.4 – Replace the Longest Chain (GET /replace_chain)

After a block was mined on node 5001 while nodes 5002 and 5003 still had only the genesis block, consensus was invoked. `replace_chain()` fetches the chain of every registered node and adopts it if it is longer and valid. This was performed on both 5002 and 5003, giving the same result.

```

1 def replace_chain(self):
2     network = self.nodes
3     longest_chain = None
4     max_length = len(self.chain)
5     for node in network:
6         response = requests.get(f'http://{node}/get_chain')
7         if response.status_code == 200:
8             length = response.json()['length']
9             chain = response.json()['chain']
10            if length > max_length and self.is_chain_valid(chain):
11                max_length = length
12                longest_chain = chain
13    if longest_chain:
14        self.chain = longest_chain
15        return True
16    return False
17
18 @app.route('/replace_chain', methods=['GET'])
19 def replace_chain():
20     replaced = blockchain.replace_chain()
21     if replaced:
22         return jsonify({'message': 'The chain was replaced by the longest one.',
23                        'new_chain': blockchain.chain}), 200
24     return jsonify({'message': 'All good. The chain is the largest one.',
25                    'actual_chain': blockchain.chain}), 200

```

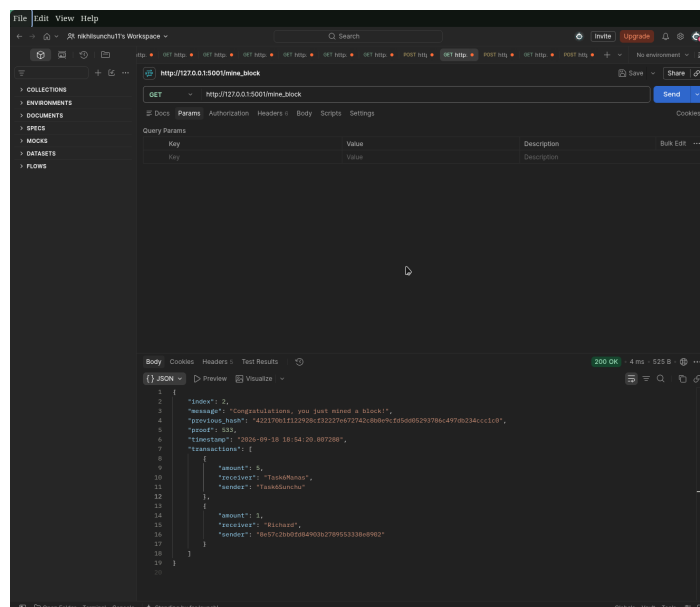


Fig. 5 – Replacing the chain on node 5002; the same result was obtained on node 5003

4.9. Task 5.5 – Final Synchronized Chain

After the consensus protocol, all nodes converged on the same longest chain, confirming that every node agrees on a single transaction history.

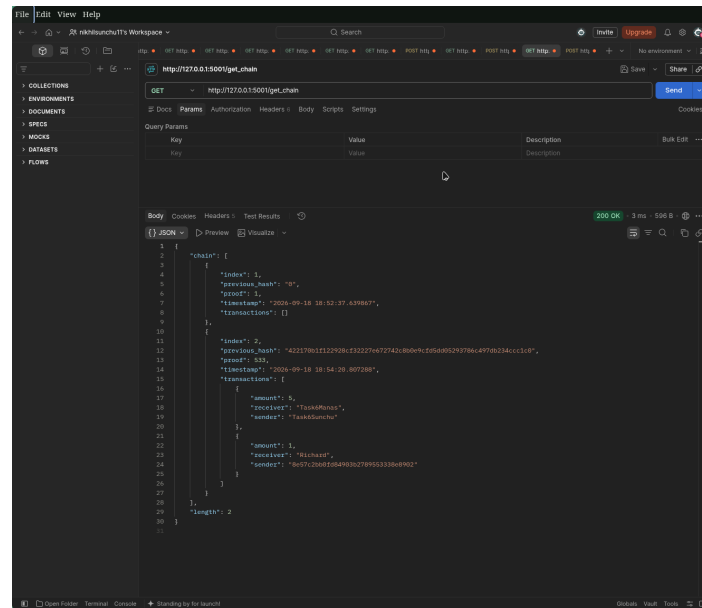


Fig. 6 – Final synchronized chain on node 5001

4.10. Task 6 – Remove Transactions After They Are Added to a Block

The code was modified so that pending transactions are cleared as soon as they are incorporated into a mined block: in `create_block()`, the current transaction pool is copied into the block and the pool is then reset, so the same transactions are never mined twice.

```
1 def create_block(self, proof, previous_hash):
2     block = {'index': len(self.chain) + 1,
3             'timestamp': str(datetime.datetime.now()),
4             'proof': proof,
5             'previous_hash': previous_hash,
6             'transactions': self.transactions}
7     self.transactions = [] # transactions removed after being added
8     self.chain.append(block)
9     return block
```

5. Conclusion

We developed a cryptocurrency using Python and implemented mining in a simulated three-node blockchain network. Each node maintained its own ledger and synchronized with peers using the Longest Chain Rule to ensure consistency. Mining was performed through Proof-of-Work, securely adding transactions and rewarding miners. This demonstrated key blockchain concepts, including decentralized consensus, transaction validation, and network synchronization.